ARTIFICIAL NEURAL NETWORK – HANDS ON

Fashion - Mnist Dataset

Importing Libraries:

Import all the required libraries for deep learning → Numpy,pandas,pydot,tensorflow,sklearn.metrics

Load the Dataset:

The Fashion - MNIST dataset is available in csv format.

fashion_train=pd.read_csv("fashion-mnist_train.csv"), fashion_test=pd.read_csv("fashion-mnist_test.csv")

The training and testing datasets are loaded into pandas DataFrames using the read_csv() function. The fashion_train DataFrame contains 60,000 labelled training images, and fashion_test contains 10,000 labelled test images. Each row represents one image, where the first column corresponds to the class label and the remaining 784 columns represent pixel intensity values of a 28×28 grayscale image.

Data Preprocessing :

The first column (label) represents the target variable.The remaining 784 columns represent pixel values.

X_train shape → (60000, 784)

In ANN the Input should be in 2D . So no reshaping needed.

Feature–Label Segregation :

To prepare the dataset for supervised learning, the target variable (label) was separated from the feature set.

- The feature matrix consists of 784 numerical attributes corresponding to flattened 28×28 grayscale image pixels.
- The target variable contains categorical labels ranging from 0 to 9, representing different clothing classes.

```
X_train_fashion = fashion_train.drop('label',axis = 1)
y_train_fashion = fashion_train['label']
X_test_fashion = fashion_test.drop('label',axis = 1)
y_test_fashion = fashion_test['label']
```

Data Visualization:

To better understand the dataset, the flattened pixel vectors were reshaped into their original 28×28 image format for visualization purposes.

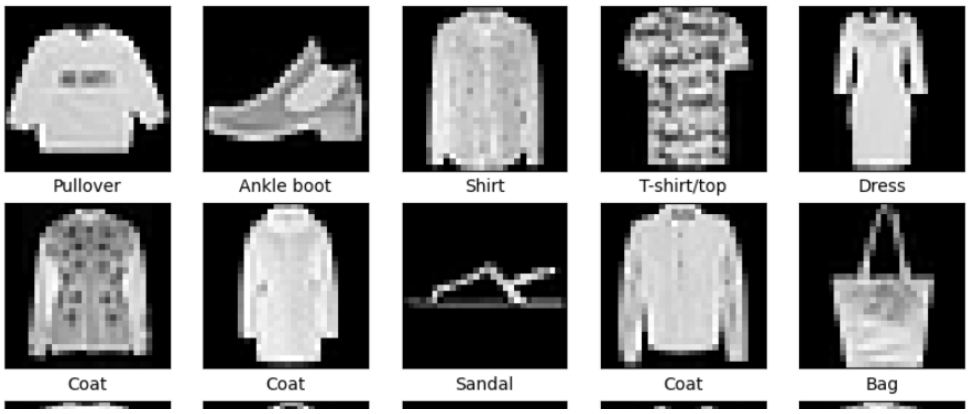
Although the ANN model uses flattened 784-dimensional vectors as input, reshaping was performed to visually inspect the images.

The numeric class labels (0–9) were mapped to their corresponding clothing category names for better interpretability.

```
#Reshaping the dataset
x_train_reshape = X_train_fashion.values.reshape(-1,28,28)
x_test_reshape = X_test_fashion.values.reshape(-1,28,28)

#Names of clothing accessories in order
col_names = ['T-shirt/top','Trouser','Pullover','Dress','Coat','Sandal','Shirt','Sneaker','Bag','Ankle boot']

#Visualizing the images
plt.figure(figsize=(10,10))
for i in range(15):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(x_train_reshape[i], cmap='gray')
    plt.xlabel(col_names[y_train_fashion[i]])
plt.show()
```



One Hot encoding of Target Variable:

The target labels were converted into one-hot encoded vectors using the to_categorical() function.

Since the problem involves 10 classes, each label was transformed into a 10-dimensional binary vector to enable multi-class classification using softmax activation.

```
y_train_fashion = to_categorical(y_train_fashion, num_classes=10)

y_test_fashion = to_categorical(y_test_fashion, num_classes=10)
```

Creating The Neural Network:

```
#Creating base neural network
model = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dropout(0.3),
    layers.BatchNormalization(),
    layers.Dense(24, activation='relu'),
    layers.Dropout(0.3),
    layers.BatchNormalization(),
    layers.Dense(24, activation='relu'),
    layers.Dropout(0.3),
    layers.BatchNormalization(),
    layers.Dense(10,activation='softmax'),
])
```

keras.Sequential([...]) -→ sequential means

The layers are stacked one after another in a linear sequence.

Input → Layer 1 → Layer 2 → Layer 3 → Output

Layer 1:

layers.Dense(128, activation='relu', input_shape=(784,))

- input_shape=(784,)
  → Each image has 784 input features (28×28 flattened).
- Dense(128)
  → 128 neurons
  → Each neuron connects to all 784 inputs.

The first hidden layer consists of **128 neurons**.

We can select number of neurons randomly. No rule to select particular number.
The number of neurons is a hyperparameter selected to provide sufficient model capacity for learning complex feature representations from the 784-dimensional input space.

Activation: ReLU:

ReLU = Rectified Linear Unit

$$ReLU(x)=max(0,x)$$

Dropout Layer:

layers.Dropout(0.3)

Randomly turns off 30% of neurons during training. Forces network not to depend on specific neurons. Dropout works only during training, not testing.

Batch Normalization

layers.BatchNormalization()

Batch Normalization is used to make training faster and more stable. It normalizes the outputs of a layer so that the values remain in a healthy range during training. This improves learning efficiency and helps the model converge more smoothly.

Layer 2:

layers.Dense(24, activation='relu')

This layer contains **24 neurons**.It receives input from the previous layer (128 outputs).It continues learning patterns from the extracted features.Dropout with a rate of 0.3 was applied to reduce overfitting and improve generalization.Batch Normalization was used to stabilize learning

Layer 3:

layers.Dense(24, activation='relu')

A third hidden layer with 24 neurons was added to allow the model to learn deeper feature representations.

Output Layer:

layers.Dense(10, activation='softmax')

10 neurons → one for each clothing category.Uses **Softmax** activation.Produces probability values.

Model Summary:

```
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense (Dense)               (None, 128)               100480

 dropout (Dropout)           (None, 128)               0

 batch_normalization (Batch  (None, 128)               512
 Normalization)

 dense_1 (Dense)             (None, 24)                3096

 dropout_1 (Dropout)         (None, 24)                0

 batch_normalization_1 (Bat  (None, 24)                96
 chNormalization)

 dense_2 (Dense)             (None, 24)                600

 dropout_2 (Dropout)         (None, 24)                0

 batch_normalization_2 (Bat  (None, 24)                96
 chNormalization)

 dense_3 (Dense)             (None, 10)                250

=================================================================
Total params: 105130 (410.66 KB)
Trainable params: 104778 (409.29 KB)
Non-trainable params: 352 (1.38 KB)
_________________________________________________________________
```

First Dense Layer:

Dense(128, input_shape=(784,))

Each of 128 neurons connects to 784 inputs.

784×128+128=100480

✓ 784 × 128 → weights
✓ 128 → biases

dropout (Dropout) (None, 128) 0 → As it does not learn weights , we have 0 trainable parameters.

batch_normalization (None, 128) 512

BatchNorm has 4 parameters per neuron:

For 128 neurons: 128×4=512

They are:

- gamma (trainable)
- beta (trainable)
- moving mean (non-trainable)
- moving variance (non-trainable)

Second Dense Layer:

dense_1 (Dense) (None, 24) 3096

Previous layer had 128 outputs.

128×24+24=3096

✓ 128 inputs
✓ 24 neurons
✓ 24 biases

Third Dense Layer:

dense_2 (Dense) (None, 24) 600

Previous layer = 24 neurons ---------- 24×24+24=600

Output Layer:

dense_3 (Dense) (None, 10) 250

Previous layer = 24 neurons ---------- 24×10+10=250

✓ 24 inputs
✓ 10 outputs
✓ 10 biases

Model Compilation:

```
#Compiling the model
model.compile(loss="categorical_crossentropy",
              optimizer="adam",
              metrics = ['accuracy'])
```

Loss Function:

Loss is a measure of how wrong the model predictions .

If predictions are far from actual values → loss is high.
If predictions are close → loss is low.

Lower loss = better model.

Optimizer Adam:

Optimizer decides how the model updates its weights to reduce loss.

After each batch:

- Model calculates loss
- Optimizer adjusts weights
- Tries to reduce error

Metrics:

Metrics are used to measure model performance in an understandable way.

Accuracy means:

Correct Predictions/Total Predictions

Model Training:

```
#Fitting the model
history=model.fit(X_train_fashion, y_train_fashion, batch_size=100, epochs=30,validation_data=(X_test_fashion, y_test_fashion))
```

```
Epoch 1/30
600/600 [==============================] - 19s 21ms/step - loss: 1.1242 - accuracy: 0.6267 - val_loss: 0.5351 - val_accuracy: 0.8235
Epoch 2/30
600/600 [==============================] - 7s 12ms/step - loss: 0.7324 - accuracy: 0.7506 - val_loss: 0.4804 - val_accuracy: 0.8337
Epoch 3/30
600/600 [==============================] - 6s 9ms/step - loss: 0.6639 - accuracy: 0.7762 - val_loss: 0.4279 - val_accuracy: 0.8532
Epoch 4/30
600/600 [==============================] - 5s 8ms/step - loss: 0.6203 - accuracy: 0.7915 - val_loss: 0.4417 - val_accuracy: 0.8516
Epoch 5/30
600/600 [==============================] - 4s 7ms/step - loss: 0.6080 - accuracy: 0.7947 - val_loss: 0.4352 - val_accuracy: 0.8538
```

```
600/600 [==============================] - 4s 7ms/step - loss: 0.5042 - accuracy: 0.8309 - val_loss: 0.4509 - val_accuracy: 0.8720
Epoch 26/30
600/600 [==============================] - 4s 6ms/step - loss: 0.4988 - accuracy: 0.8317 - val_loss: 0.4355 - val_accuracy: 0.8700
Epoch 27/30
600/600 [==============================] - 4s 7ms/step - loss: 0.4943 - accuracy: 0.8342 - val_loss: 0.4209 - val_accuracy: 0.8704
Epoch 28/30
600/600 [==============================] - 4s 6ms/step - loss: 0.4983 - accuracy: 0.8333 - val_loss: 0.4340 - val_accuracy: 0.8713
Epoch 29/30
600/600 [==============================] - 4s 7ms/step - loss: 0.4993 - accuracy: 0.8322 - val_loss: 0.4277 - val_accuracy: 0.8717
Epoch 30/30
600/600 [==============================] - 4s 7ms/step - loss: 0.4932 - accuracy: 0.8355 - val_loss: 0.4271 - val_accuracy: 0.8737
```

X_train_fashion →Training Input

y_train_fashion →Training Output

batch_size = 100 → The model processes **100 images at a time**. After each batch It calculates loss and Updates weights.

Epochs = 30 → One full pass through the entire training dataset. → 60000 images 30 times.

validation_data=(X_test_fashion, y_test_fashion) →

After each epoch, The model is tested on unseen test data And Validation loss & accuracy are calculated.

Evaluation:

```
test_loss_fashion, test_acc_fashion = model.evaluate(X_test_fashion, y_test_fashion)
```

```
313/313 [==============================] - 16s 13ms/step - loss: 0.4271 - accuracy: 0.8737
```

model.evaluate(X_test_fashion, y_test_fashion) – After training the model makes predictions on the **test dataset** to calculate loss and accuracy.

test_loss_fashion – Final error value on Test data
test_acc_fashion – Accuracy on Test Data

Prediction:

```
#Predicting the labels-Fashion
y_predict_fash = model.predict(X_test_fashion)
y_predict_fash=np.argmax(y_predict_fash, axis=1)
y_test_fash_eval=np.argmax(y_test_fashion, axis=1)
```

```
313/313 [==============================] - 4s 5ms/step
```

The trained model makes predictions on test data.

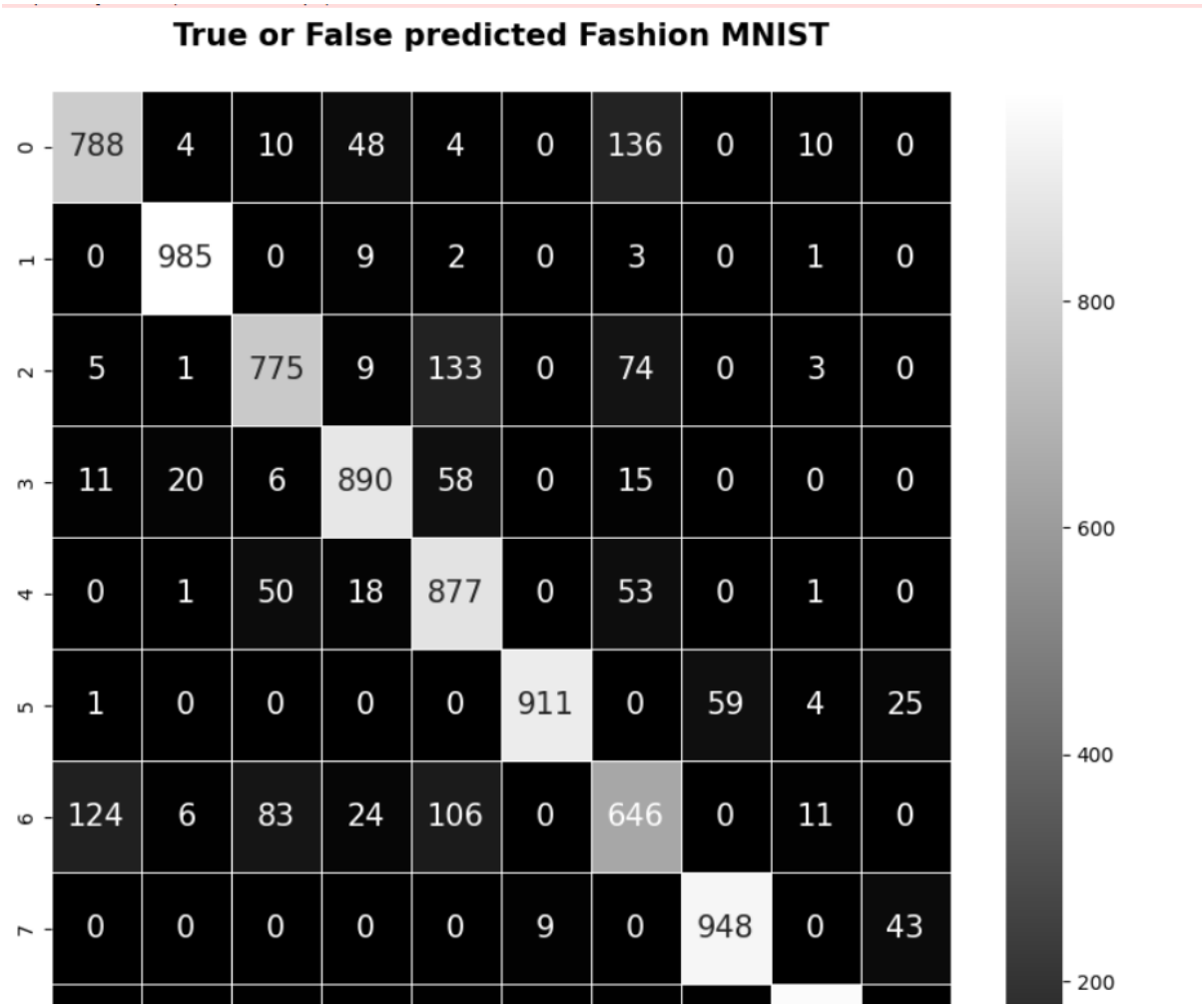
As we used Softmax for last layer, the output will be **probabilities**.

np.argmax(y_predict_fash, axis=1) → argmax() will return the index (position) of highest value.

Similarly, the one-hot encoded true labels were converted back to their original numeric class representation using argmax().

Confusion Matrix Visualization:

```
con_mat=confusion_matrix(y_test_fash_eval,y_predict_fash)
plt.style.use('seaborn-deep')
plt.figure(figsize=(10,10))
sns.heatmap(con_mat,annot=True,annot_kws={'size': 15},linewidths=0.5,fmt="d",cmap="gray")
plt.title('True or False predicted Fashion MNIST\n',fontweight='bold',fontsize=15)
plt.show()
```



Here confusion matrix compares actual labels with predicted labels.

Diagonal values are correct predictions others are wrong.

A heatmap was used to visually represent the matrix, enabling easier interpretation of model performance across all 10 classes.

Classification Report:

```
from sklearn.metrics import classification_report

print(classification_report(y_test_fash_eval,y_predict_fash))
```

```
              precision    recall  f1-score   support

           0       0.85      0.79      0.82      1000
           1       0.97      0.98      0.98      1000
           2       0.83      0.78      0.80      1000
           3       0.89      0.89      0.89      1000
           4       0.74      0.88      0.80      1000
           5       0.98      0.91      0.95      1000
           6       0.69      0.65      0.67      1000
           7       0.90      0.95      0.92      1000
           8       0.97      0.97      0.97      1000
           9       0.93      0.95      0.94      1000

    accuracy                           0.87     10000
   macro avg       0.87      0.87      0.87     10000
weighted avg       0.87      0.87      0.87     10000
```

It compares the actual labels with predicted labels and print performance report for all classes.

Precision – How many predicted values of a class were correct

Recall – How many actual values were correctly identified

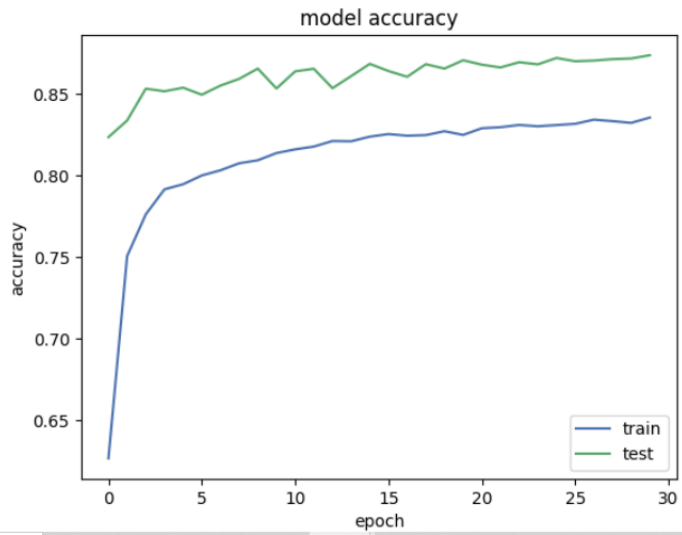
F1-score – Balance between Precision and Recall

Strong Performing Classes – Class 1, Class 5, Class 8

Weakest Class – Class 6

Accuracy Graph:

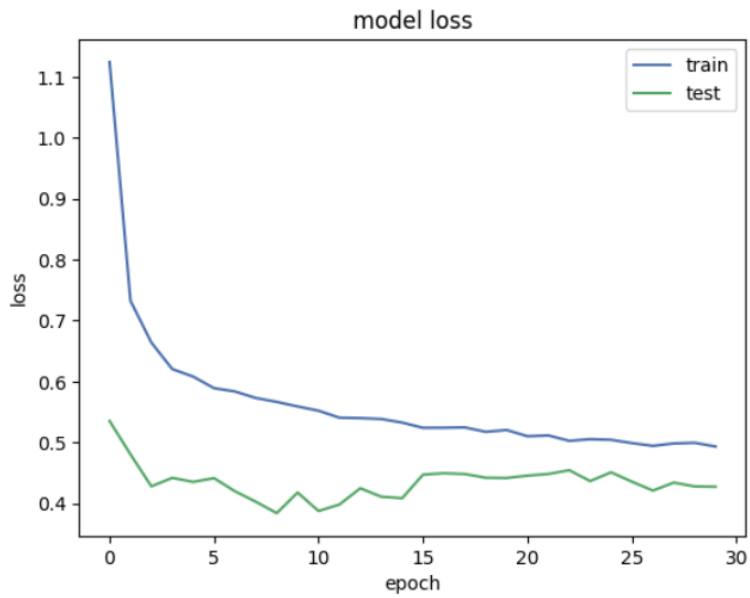
```
# summarize history for accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='best')
plt.show()
```



Validation accuracy is higher than training accuracy.

Loss Graph:

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='best')
plt.show()
```



Validation loss is lower than training loss.

Checking for single image:

```
#tf.expand_dims(X_test_digit[0])
y_predict = model.predict(X_test_fashion.loc[[0],:].values)
y_predict=np.argmax(y_predict, axis=1) # Here we get the index of maximum value in the encoded vector
y_test_digit_eval=np.argmax(y_test_fashion, axis=1)
```

```
1/1 [==============================] - 1s 711ms/step
```

X_test_fashion.loc[[0], :] → 0 – row label, : - All columns

np.argmax(y_predict, axis=1) → Here it will change the probabilities into class as output

y_test_digit_eval = np.argmax(y_test_fashion, axis=1) → This converts all one-hot test labels back to class numbers.

```
#Names of clothing accessories in order
col_names = ['T-shirt/top','Trouser','Pullover','Dress','Coat','Sandal','Shirt','Sneaker','Bag','Ankle boot']

#Visualizing the images
plt.figure(figsize=(10,10))

plt.imshow(x_train_reshape[0], cmap='gray')
plt.xlabel("Actual:{},Pred:{}".format(col_names[np.argmax(y_test_fashion[0])],col_names[y_predict[0]]))
plt.show()
```

Here we defined col names - clothing names as our model predicts numbers (0–9).

plt.xlabel("Actual:{} , Pred:{}".format(col_names[np.argmax(y_test_fashion[0])],col_names[y_predict[0]]))

np.argmax(y_test_fashion[0]) - Using argmax to get the value from one-hot encoded

col_names[2] → Pullover

col_names[y_predict[0]] – If model predicted 2 Then both are same values.

This visualization helps interpret model predictions and evaluate correctness at an individual sample level.